Grid Features Validation Packet -- 8Z3ESD

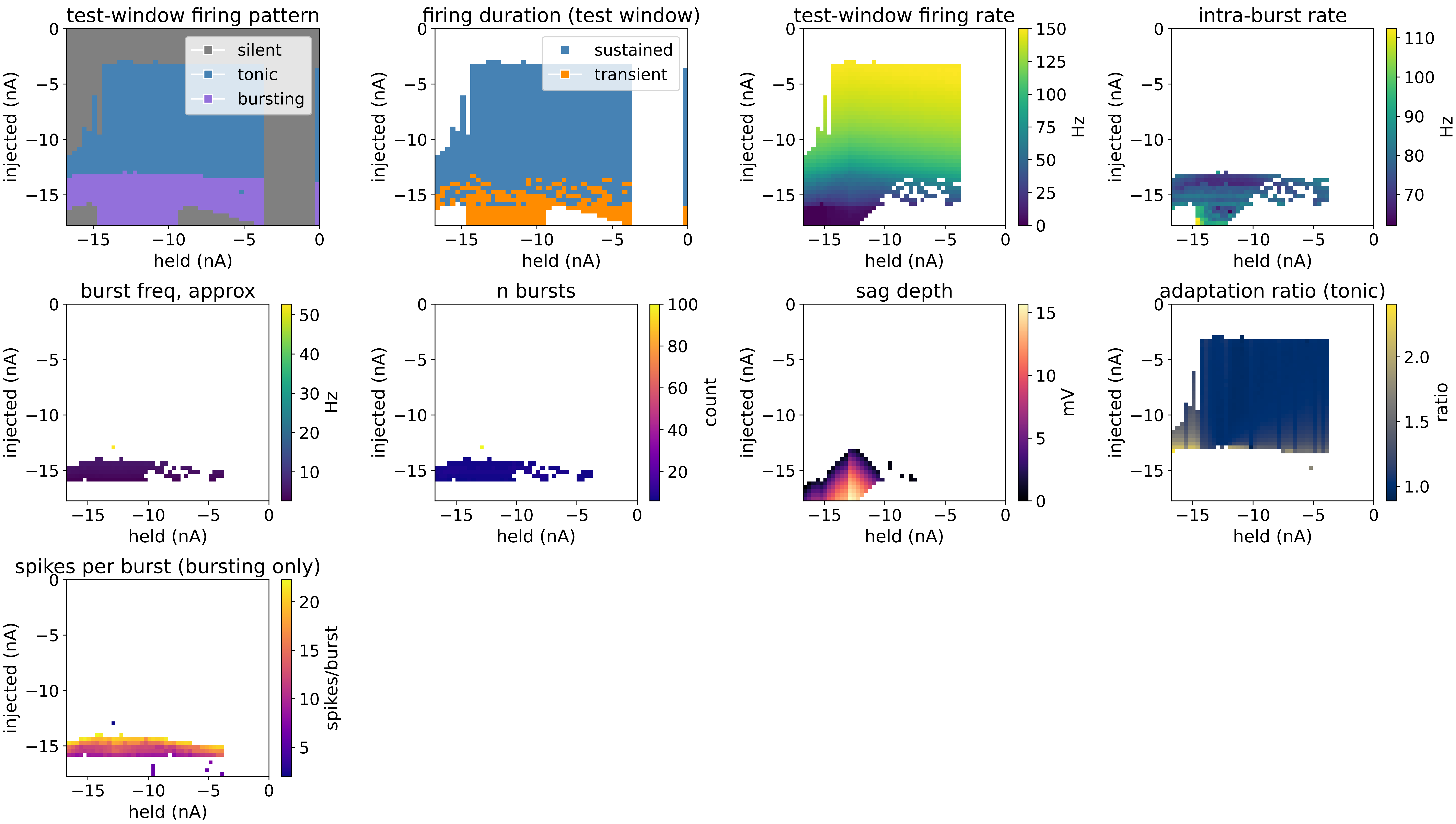
Every trace in this packet is simulated output from a fitted, multi-conductance single-compartment model of this cell, integrated deterministically -- not a raw current-clamp recording. What is being evaluated here is the feature-extraction methodology applied on top of that model: spike detection, burst/tonic classification, post-inhibitory rebound detection, sag depth, and spike-frequency adaptation. Each figure below reruns the actual classification algorithm on a real simulated trace and marks what it found directly on the trace.

```
Silencing threshold: -16.75 nA   |   2500 points sampled across the current grid
Burstiness index: 0.224
Firing-rate/current slope: not available (no tonic points to fit)
```

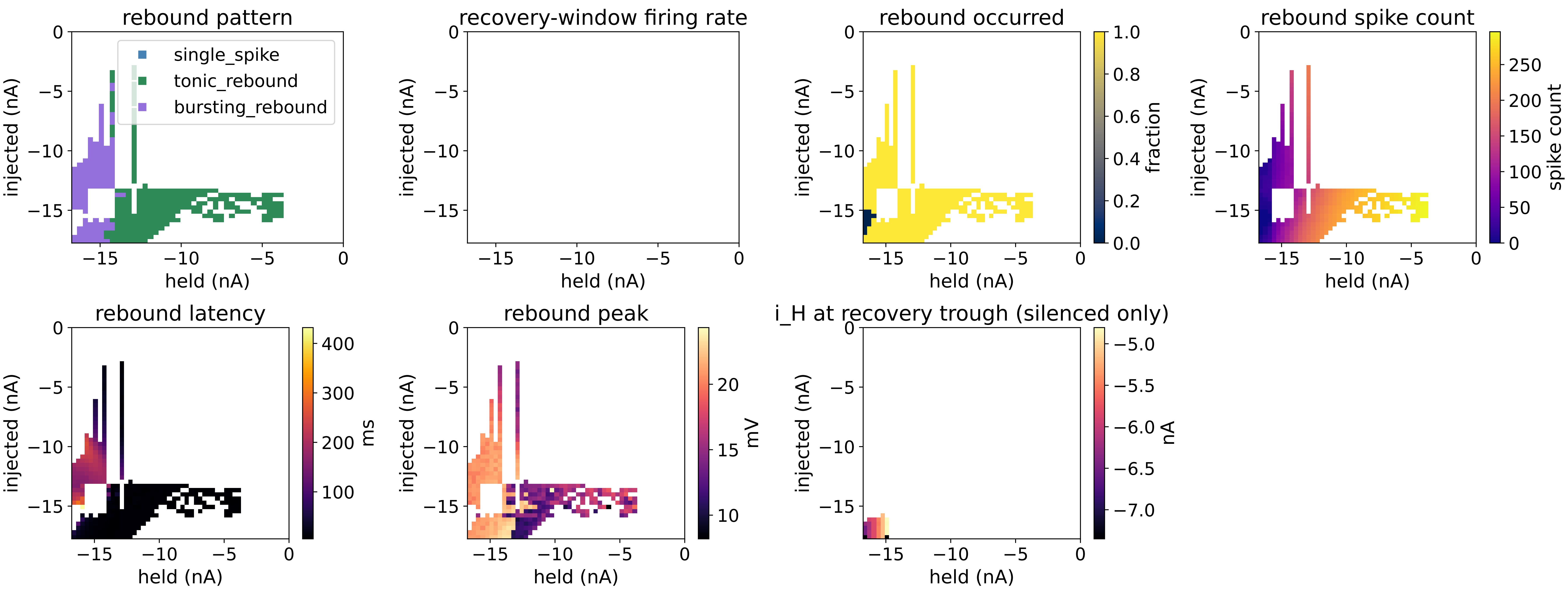
Contents:

1. Feature overview: 8Z3ESD's full current grid

TEST PHASE



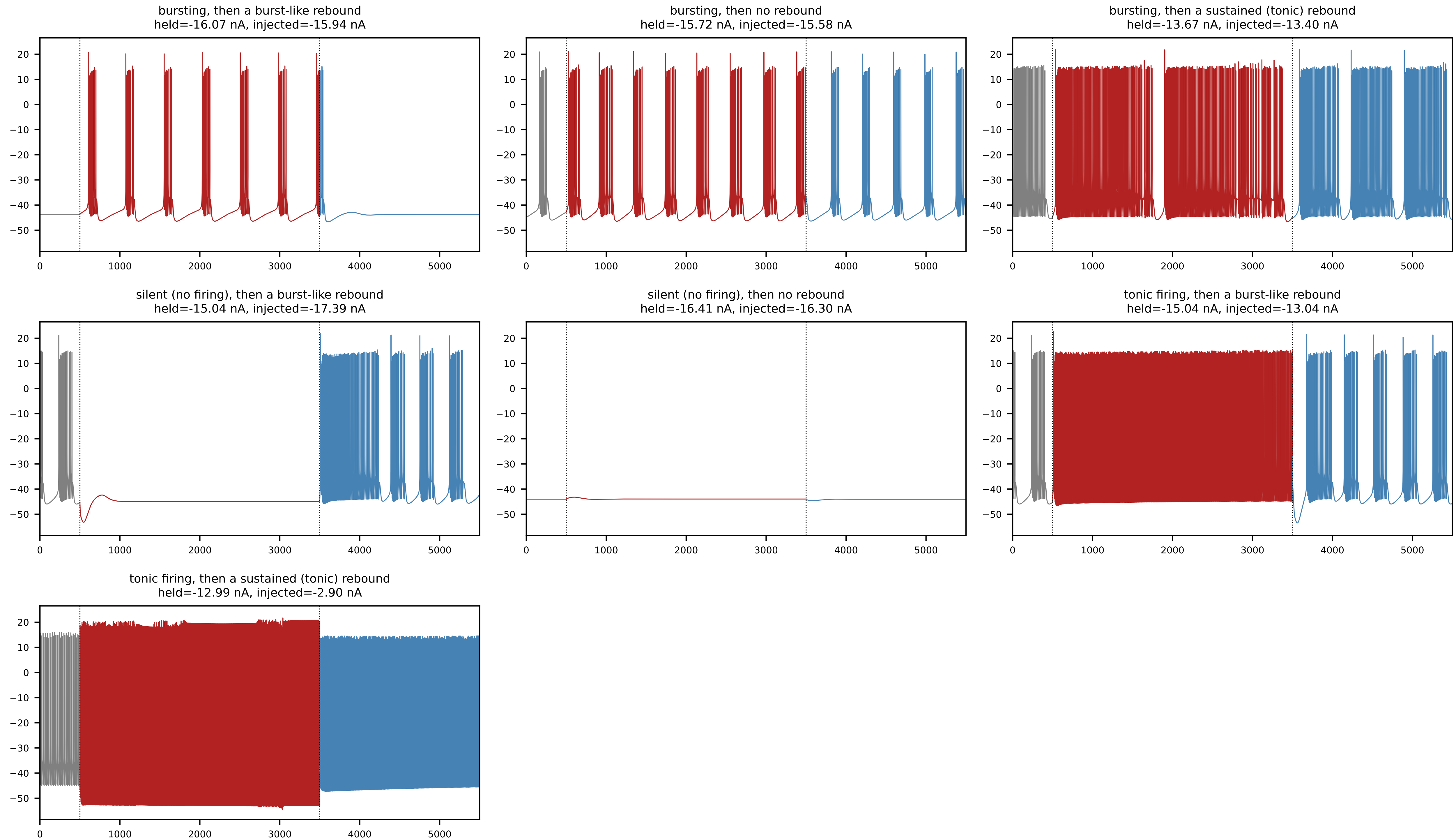
RECOVERY PHASE



Every panel below is computed from the same 2500-point grid of held and injected current levels: which pattern each point fires in and whether it rebounds after release from hyperpolarization (top row), firing rate and burst structure (second row), rebound spike count, latency, and peak voltage (third row), and sag depth, adaptation, and burst size (bottom row). Two single-number summaries condense the whole grid: a burstiness index of 0.224 (the fraction of points classified bursting) and a firing-rate-vs-current slope that is not available (no tonic points to fit). The rest of this packet does not add new results -- it re-derives individual pieces of this same grid from real simulated traces so each one can be checked by eye against the panel it feeds.

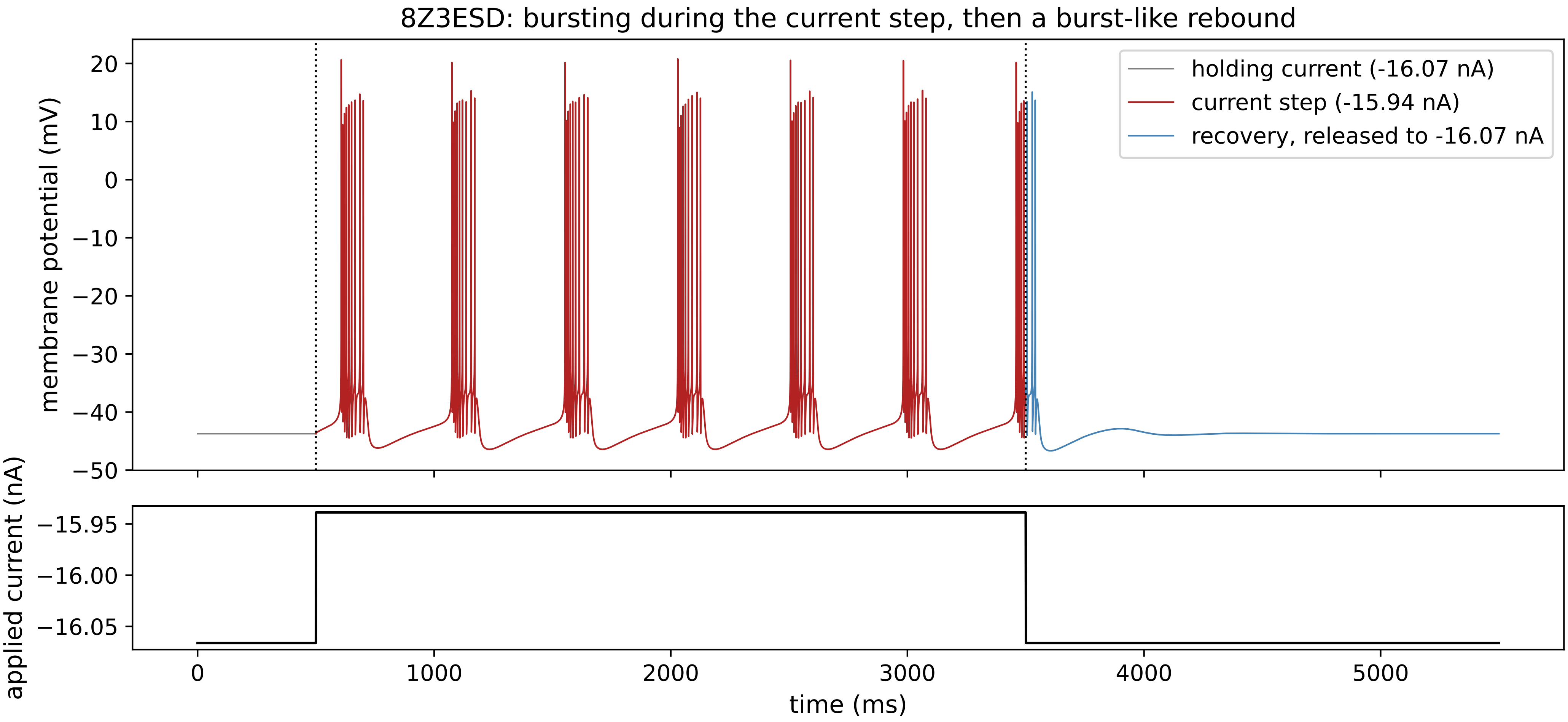
```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> cell_grid_features.pkl (run_held_injected_grid.build_cell_grid_features_fig(), no resimulation)
```

2. Representative traces

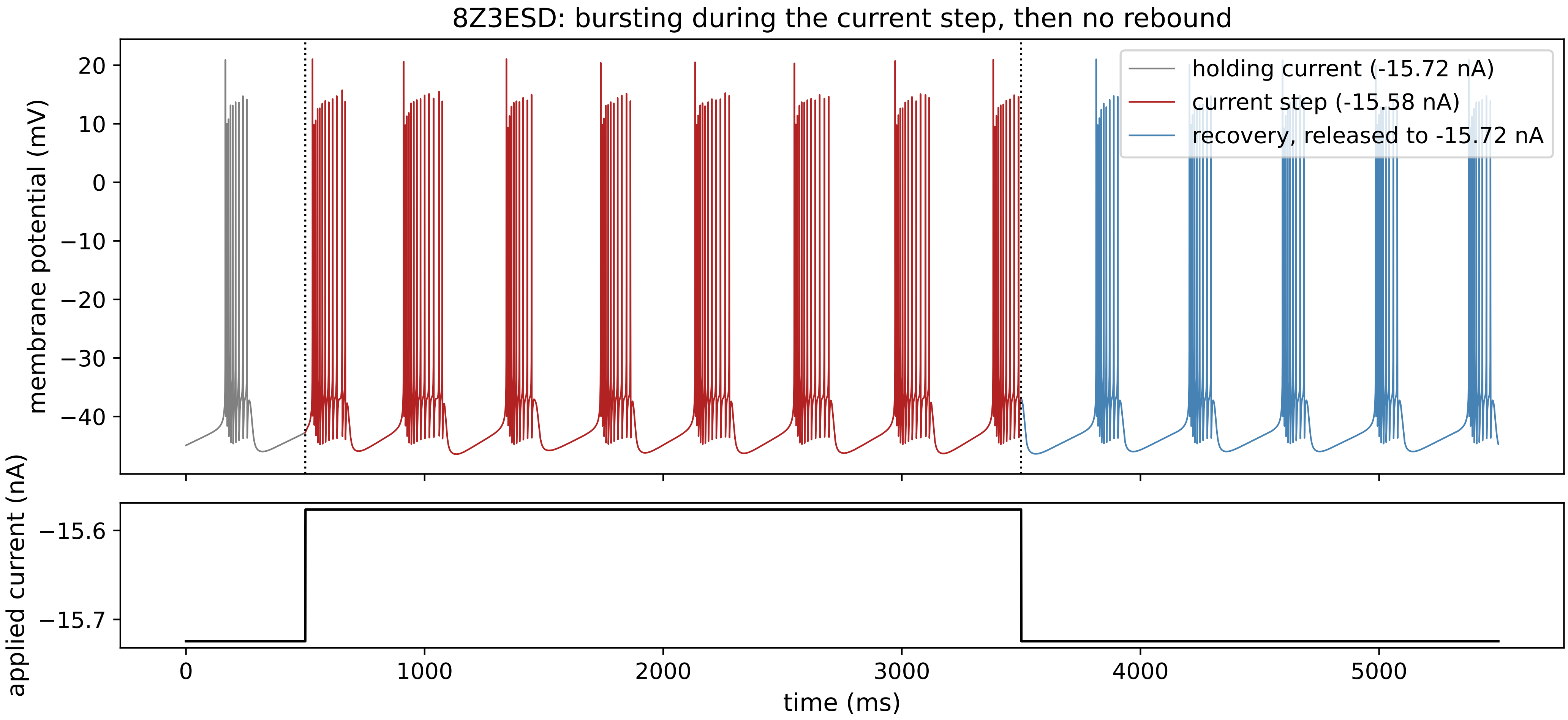


One representative trace for each distinct combination of response-to-current-step pattern and rebound pattern observed across this cell's full 2500-point grid of held and injected current levels: gray is the held baseline before the current step, red is the response during the current step, and blue is the recovery period after release back to the held level. Each combination is represented by its most clearly-expressed example among the grid points that produced it. Every panel shares the same voltage (-58 to 26 mV) and time (0-5500 ms) axes so amplitude and duration are directly comparable by eye -- without this, a small subthreshold wobble and a full-amplitude spike train would each be independently rescaled to fill the same panel and look equally prominent.

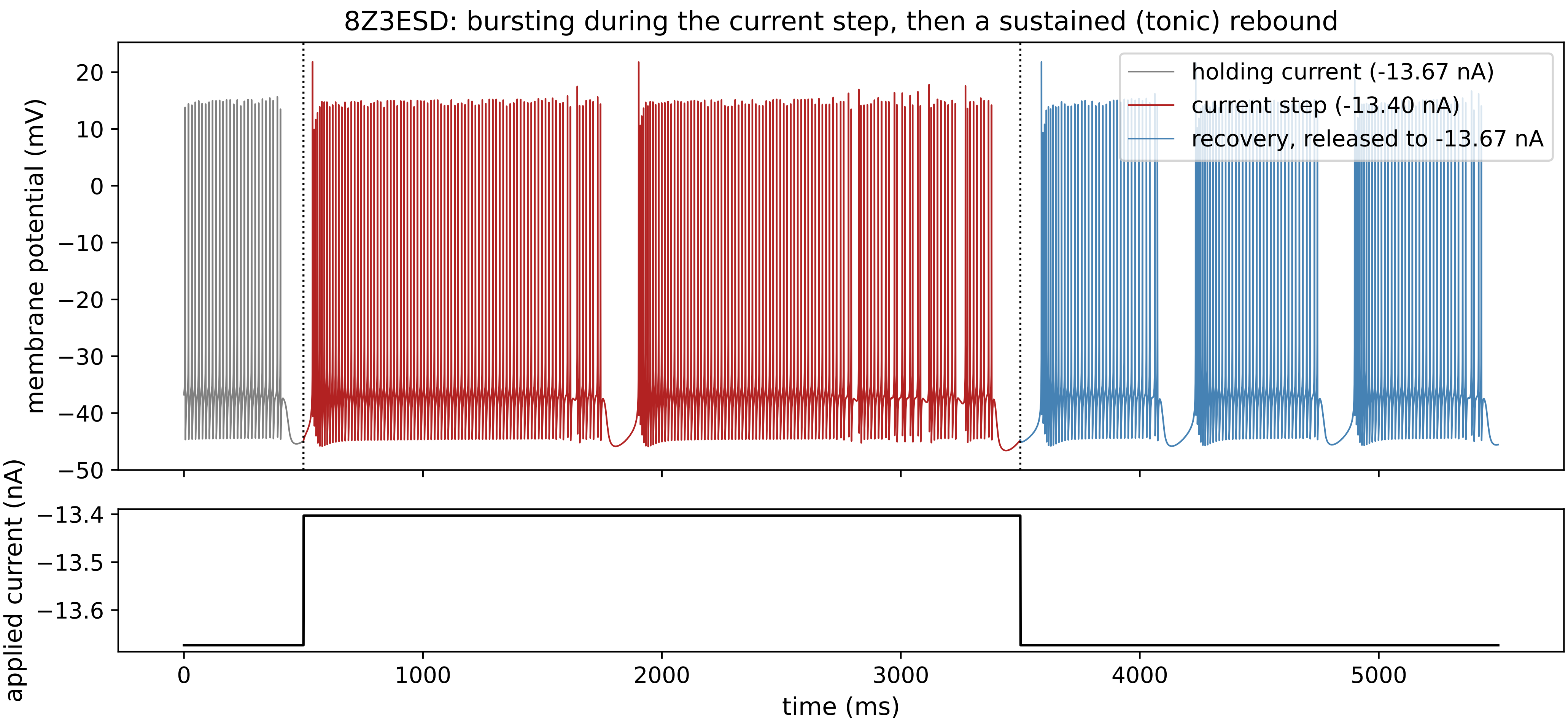
```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point()
```



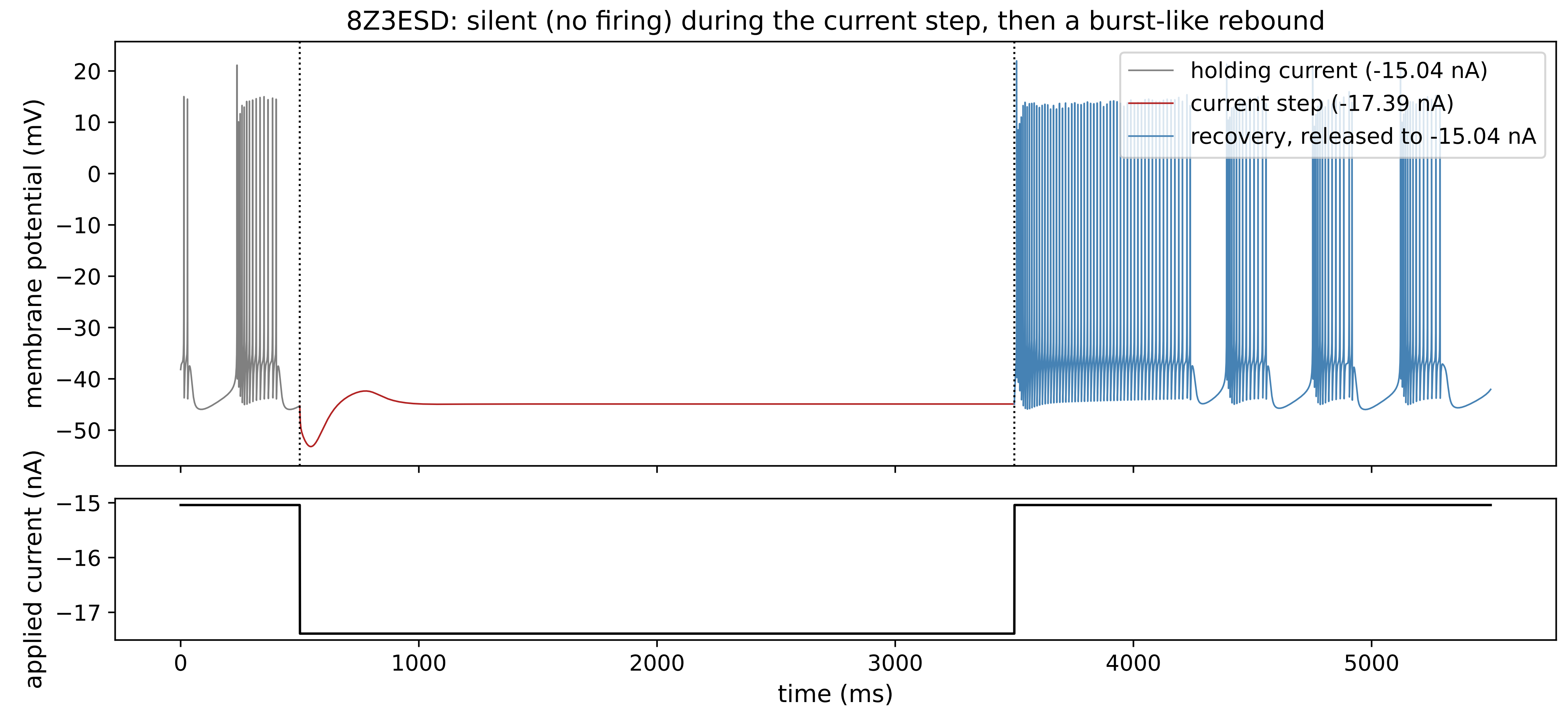
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()



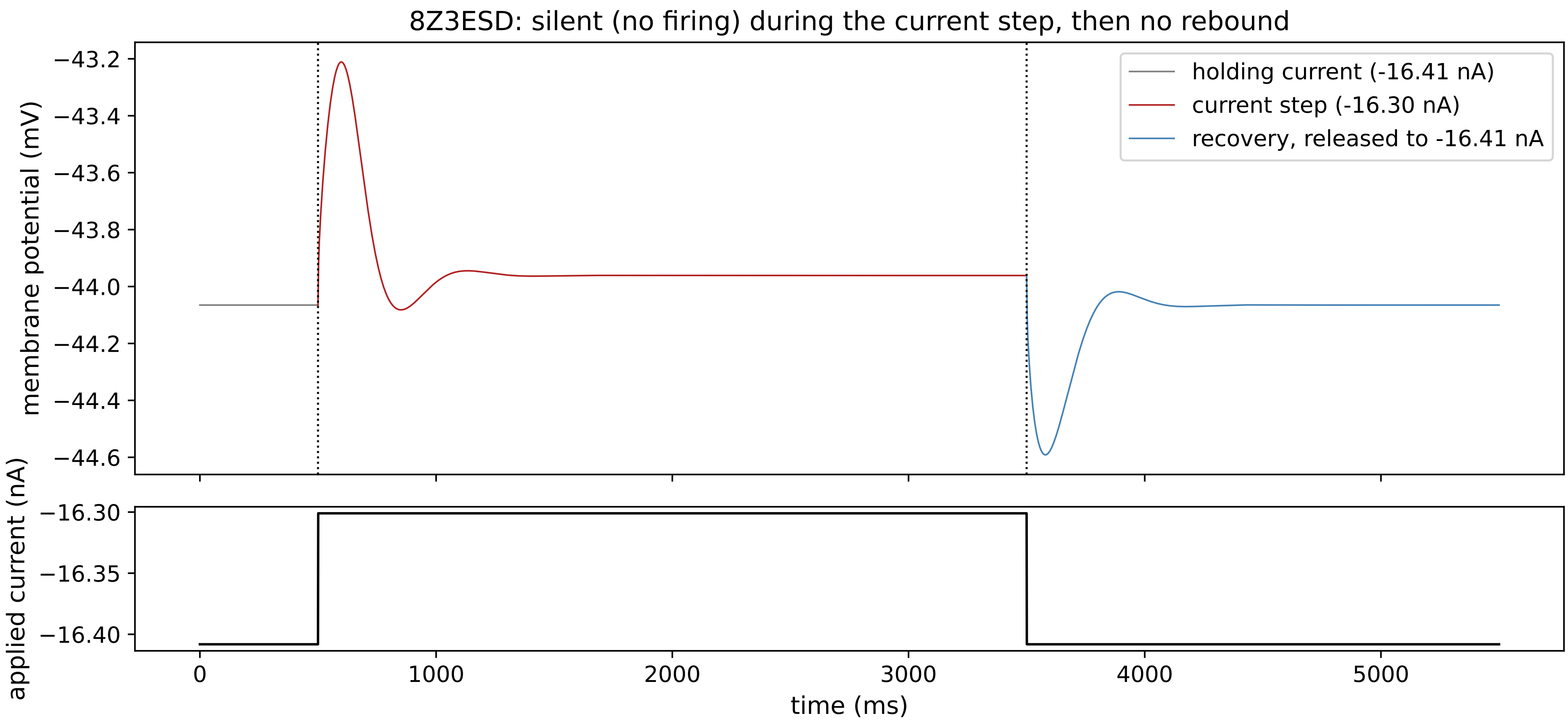
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()




```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()
```

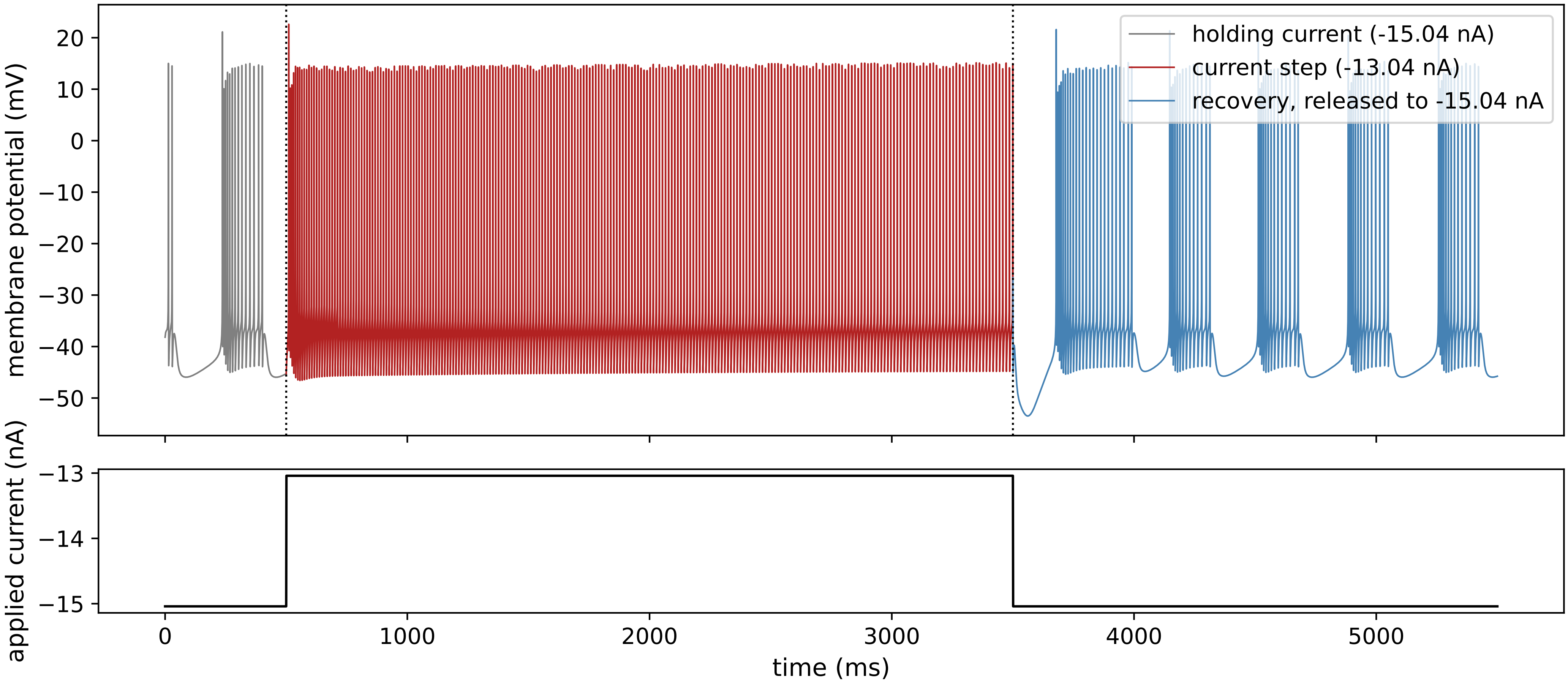


python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

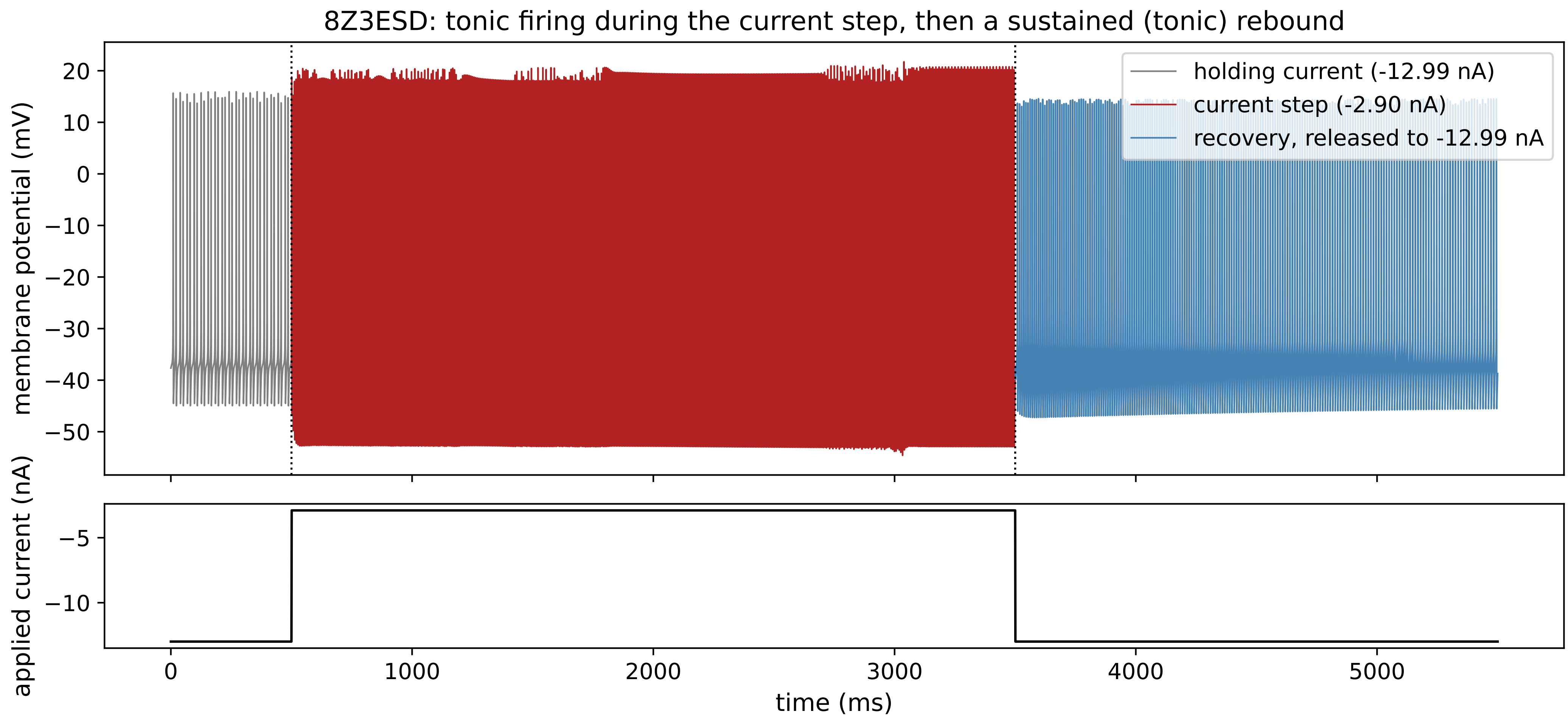


python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

8Z3ESD: tonic firing during the current step, then a burst-like rebound

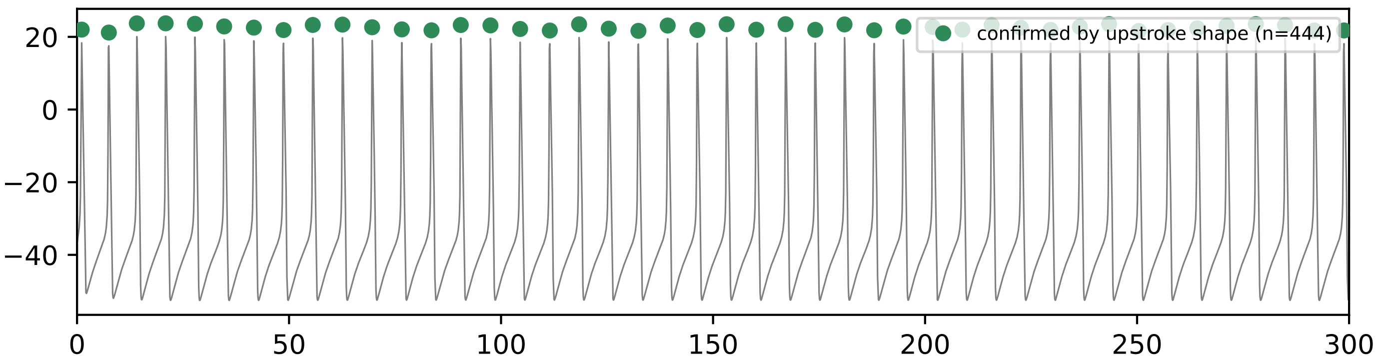



```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()
```

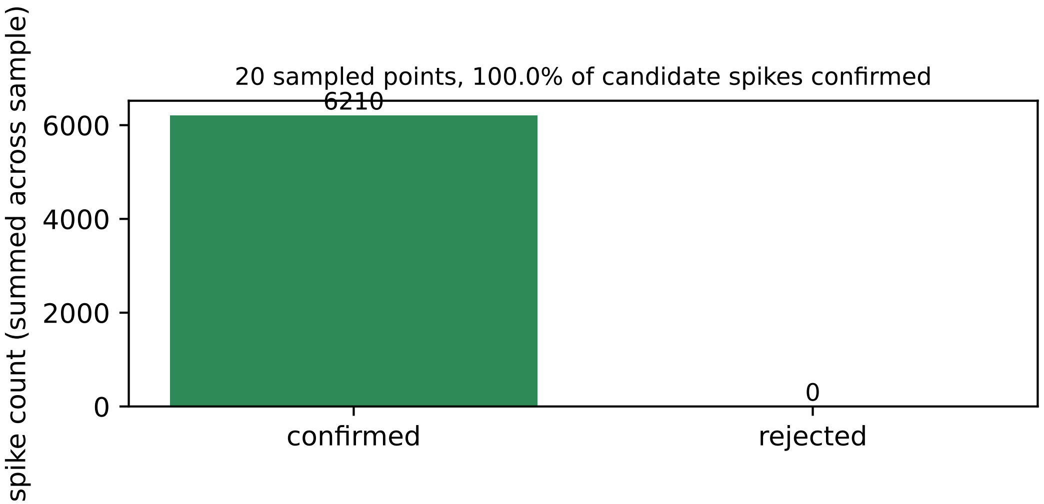


python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> plot_example_traces.select_exemplar_points() + resimulate_point() -> plot_example_traces.build_example_trace_figure()

5. Independent cross-check of spike detection by upstroke shape



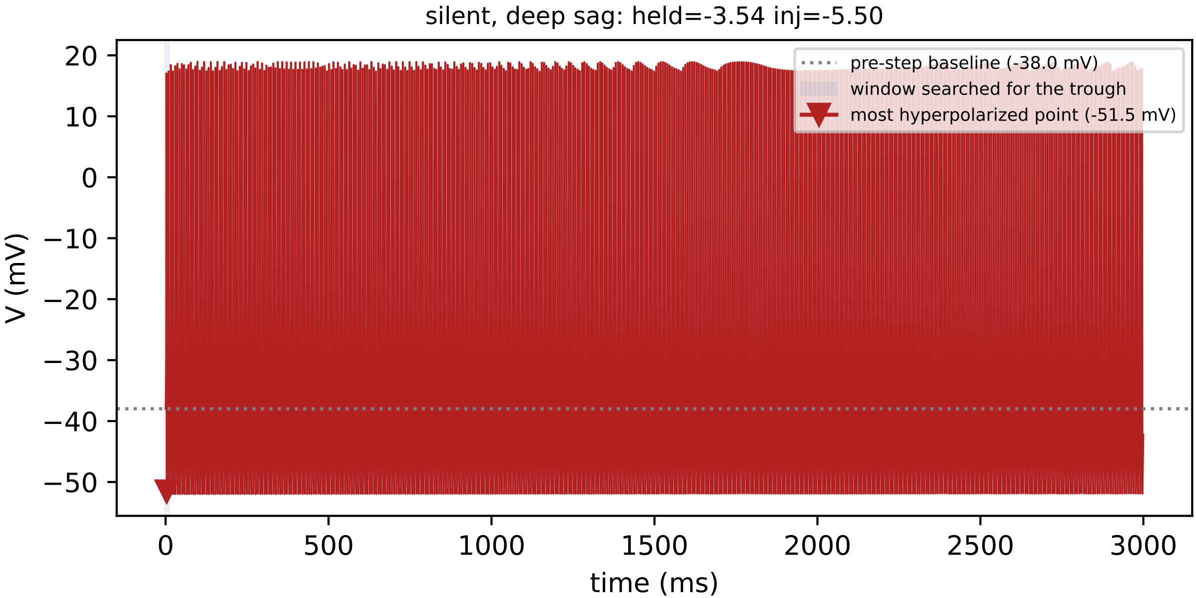
As an independent check, each candidate spike was also required to show a fast upstroke -- a rate of voltage rise of at least 10 mV/ms within 20 ms before the peak (the convention used by Bean, 2007) -- rather than amplitude alone. 444 of 444 candidate spike(s) were confirmed by this stricter test; the two methods agree on every spike in this window.



As an independent check on a random sample of 20 grid points from this cell, every spike detected by amplitude alone was additionally required to show a fast upstroke in voltage (rather than amplitude alone) before being counted, an independent detection criterion not used for classification elsewhere in this report. Agreement close to 100% shows that the simpler amplitude-based detector used throughout is not missing any spikes that a shape-based check would catch.

python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> find_silencing_threshold.detect_spikes_dvdt_confirmed()

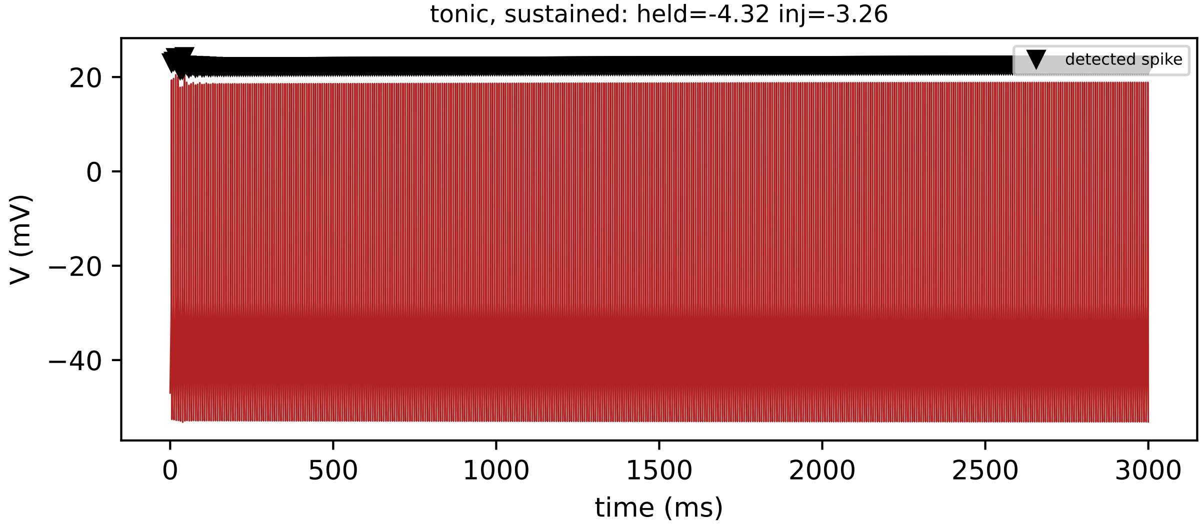
10. Sag depth

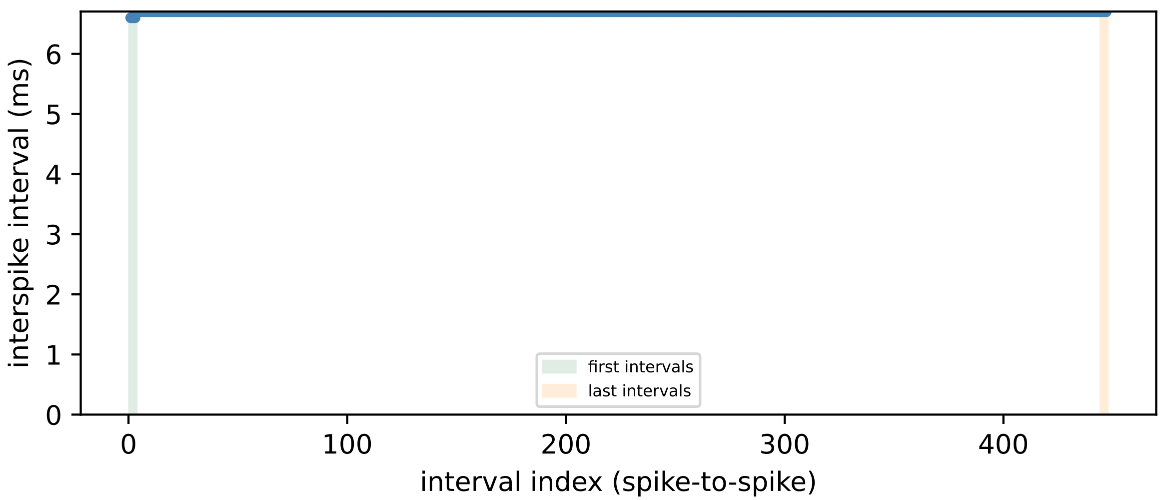


Sag depth is the difference between the voltage just before the current step (-38.0 mV) and the most hyperpolarized point reached afterward (-51.5 mV), giving 13.6 mV of sag -- a hallmark of the hyperpolarization-activated current (Ih) partially repolarizing the cell during the step. The trough was found by searching stopping early at the first spike, 9 ms in.

```
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_pre_spike_sag_trough()
```

11. Spike-frequency adaptation

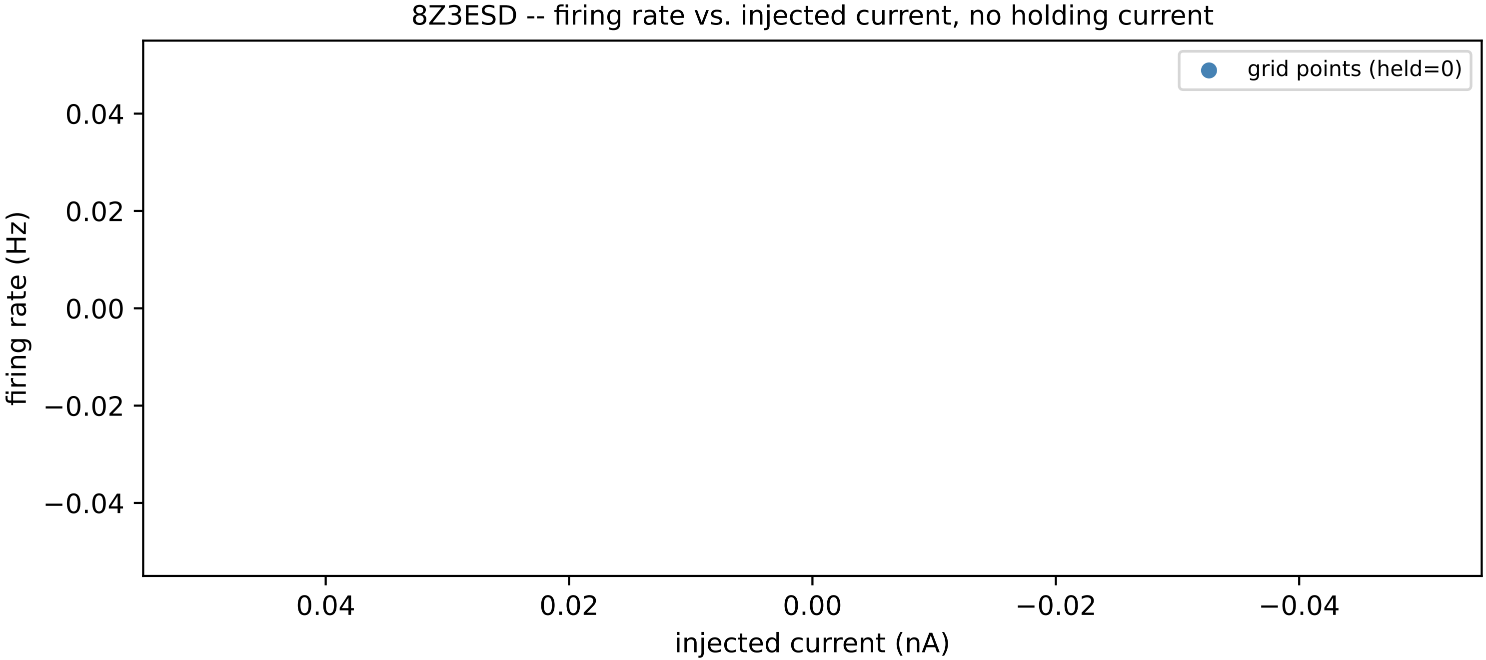




The adaptation ratio -- the mean of the last 3 interspike intervals divided by the mean of the first 3 -- is 1.02. Firing stays roughly constant over this window.

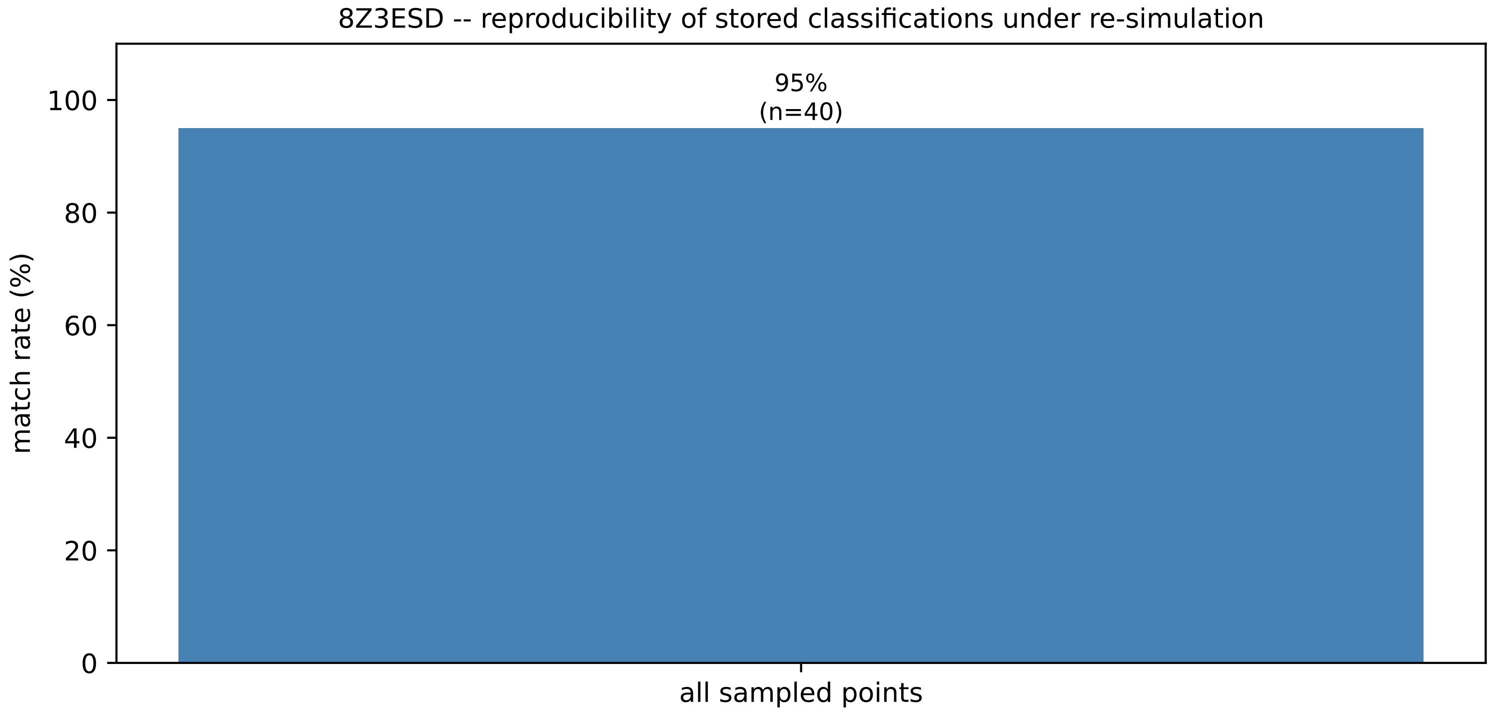
python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> resimulate_point() -> run_held_injected_grid.compute_adaptation_ratio()

12. Firing rate vs. injected current



0 non-silent points with a clearly-classified firing pattern, sampled across injected current with no holding current applied (points still below firing threshold, which would otherwise bias the linear fit toward zero, are excluded). This firing-rate/current slope is one of the features used to characterize this cell in the cross-cell comparison; here it is plotted directly against the data it was fit to.

13. Reproducibility check



40 random points from this cell's grid were re-simulated independently and compared against the classification stored from the original sweep. Match rate: 95%. Except at zero holding current, where the cell's own steady-state limit cycle is reused exactly, each holding-current level is reached by integrating forward from whichever already-settled level is numerically nearest -- this model is known to show path-dependent (hysteretic) settling near dynamical transitions, so a re-simulation that approaches a borderline point along a different path than the original sweep can occasionally land on a different classification right at that boundary.

python src/generate_validation_packet.py --cells DFJ1K3 8Z3ESD MXIC4S
source: cell_held_injected_grid.pkl -> cached test_pattern/rebound_pattern vs. fresh resimulate_point() results